

**UNIVERSITY INSTITUTE OF ENGINEERING &
TECHNOLOGY
MAHARSHI DAYANAND UNIVERSITY ROHTAK**

PRACTICAL FILE

SUBJECT: COMPILER DESIGN (PCC-CSE-302G) B.TECH (CSE)

6TH SEM



SUBMITTED TO

Ms. Jyoti
(Research Scholar)
Deptt of Computer Science
MDU, Rohtak

SUBMITTED BY

CHAITANYA YADAV
ROLL NO- 2214051
CSE-A

INDEX

SERIAL NO	NAME OF PROGRAM	REMARKS
1	Program for Token separation with a given expression	
2	Program for Token separation with a given file	
3	Program for Lexical analysis using LEX tools	
4	Program to find FIRST in context free grammer	
5	Program to find whether a given string is keyword or not	
6	Program to check whether a given string is constant or variable	
7	Program to identify whether a given line is a comment or not	
8	Program to check whether a given identifier is valid or not	
9	Program to recognize strings under 'a', 'a*b+', 'abb'	
10	Program to simulate lexical analyser for validating operations	
11	Program for implementation of Operator Precedence Parser	
12 A	Program for implementation of calculator using YACC tool	
12 B	Program for implementation of Recursive Descent Parser using LEX tool	
12 C	Program for implementation of LALR Parser	

Program – 1

Program for Token separation with a given expression

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
#include<ctype.h>
void main()
{
char exp[50]="\0",con[50]="\0",kwd[50]="\0",id[50]="\0",sym[50]="\0",opr[50]="\0";
char key[6][10]={"if","for","do","while","int","float"}; char ch;
char ptr[10][10]={"\0"}; int
i=0,j=0,k=-1,n=-1,p=-1,s=-1;
clrscr();
puts("Enter the expression for lexical analysis"); gets(exp);
puts("\nThe tokens are"); do
{
ch=exp[i];
if(isalpha(ch))
{
k=-1;
ptr[++n][++k]=ch;
i++; ch=exp[i];
if(isalpha(ch) || isdigit(ch))
{
while(isalpha(ch) || isdigit(ch))
{
ptr[n][++k]=ch;
i++;
ch=exp[i];
}
while(j<6)
{
if(strcmp(key[j],ptr[n])==0)
{
ptr[n][++k]=' ';
strcat(kwd,ptr[n]);
break;
}
if(j==5)
{
ptr[n][++k]=' ';
strcat(id,ptr[n]);
```

```
}
j++;
}
}
else {
ptr[n][++k]=' ';
strcat(id,ptr[n]);
}
i--;
ch=exp[i];
j=0; }
else if(isdigit(ch))
{
k=-1;
ptr[++n][++k]=ch;
i++; ch=exp[i];
if(isdigit(ch))
{
while(isdigit(ch))
{
ptr[n][++k]=ch;
i++; ch=exp[i];
} }
i--;
ch=exp[i];
ptr[n][++k]=' ';
strcat(con,ptr[n]);
}
else if(ch=='+' || ch=='-' || ch=='*' || ch=='/' || ch=='%' || ch=='>' || ch=='<' || ch=='=' || ch=='!')
{
opr[++p]=ch;
i++; ch=exp[i];
if(ch=='+' || ch=='-' || ch=='*' || ch=='/' || ch=='%' || ch=='>' || ch=='<' || ch=='=' || ch=='!')
{
opr[++p]=ch;
}
else
{ i--;
ch=exp[i];
opr[++p]=' '; }
}
else
{
```

```
sym[++s]=ch;
sym[++s]=' '; }
i++;
}while(exp[i]!='\0');
puts("\nKeyword:");
puts(kwd);
puts("\nIdentifier:");
puts(id);
puts("\nConstant:");
puts(con);
puts("\nOperator:");
puts(opr);
puts("\nSymbol:");
puts(sym); getch(); }
```

OUTPUT:

```
main.c: in function 'main':
main.c:14:1: warning: 'gets' is deprecated [-Wdeprecated-declarations]
  14 | gets(exp);
      | ^~~~
In file included from main.c:1:
/usr/include/stdio.h:577:14: note: declared here
 577 | extern char *gets (char *__s) __wur __attribute_deprecated__ ;
      |                  ^~~~
/usr/bin/ld: /tmp/ccjj225w.o: in function 'main':
main.c:(.text+0x2ad): warning: the 'gets' function is dangerous and should not be used.
Enter the expression for lexical analysis
hello world

The tokens are

Keyword:

Identifier:
hello world

Constant:

Operator:

Symbol:

...Program finished with exit code 0
Press ENTER to exit console.
```

Program – 2

Program for Token separation with a given file

```
#include <stdbool.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

// Returns 'true' if the character is a DELIMITER. bool
isDelimiter(char ch)
{
    if (ch == ' ' || ch == '+' || ch == '-' || ch == '*' ||
        ch == '/' || ch == ',' || ch == ';' || ch == '>' || ch
        == '<' || ch == '=' || ch == '(' || ch == ')' || ch
        == '[' || ch == ']' || ch == '{' || ch == '}') return
        (true); return (false);
}

// Returns 'true' if the character is an OPERATOR. bool
isOperator(char ch)
{
    if (ch == '+' || ch == '-' || ch == '*' ||
        ch == '/' || ch == '>' || ch == '<' ||
        ch == '=') return (true); return
        (false);
}

// Returns 'true' if the string is a VALID IDENTIFIER. bool
validIdentifier(char* str)
{
    if (str[0] == '0' || str[0] == '1' || str[0] == '2' ||
        str[0] == '3' || str[0] == '4' || str[0] == '5' ||
```

```
str[0] == '6' || str[0] == '7' || str[0] == '8' ||
str[0] == '9' || isDelimiter(str[0]) == true)
return (false); return (true);
}
// Returns 'true' if the string is a KEYWORD.
bool isKeyword(char* str)
{
if (!strcmp(str, "if") || !strcmp(str, "else") || !strcmp(str,
"while") || !strcmp(str, "do") ||
!strcmp(str, "break") ||
!strcmp(str, "continue") || !strcmp(str, "int")
|| !strcmp(str, "double") || !strcmp(str, "float")
|| !strcmp(str, "return") || !strcmp(str, "char")
|| !strcmp(str, "case") || !strcmp(str, "char")
|| !strcmp(str, "sizeof") || !strcmp(str, "long")
|| !strcmp(str, "short") || !strcmp(str, "typedef")
|| !strcmp(str, "switch") || !strcmp(str, "unsigned")
|| !strcmp(str, "void") || !strcmp(str, "static")
|| !strcmp(str, "struct") || !strcmp(str, "goto"))
return (true); return (false);
}
// Returns 'true' if the string is an INTEGER.
bool isInteger(char* str)
{
int i, len = strlen(str);
if (len == 0) return (false); for (i = 0; i <
len; i++) { if (str[i] != '0' && str[i] != '1' &&
str[i] != '2'
&& str[i] != '3' && str[i] != '4' && str[i] != '5'
```



```
&& str[i] != '6' && str[i] != '7' && str[i] != '8'
&& str[i] != '9' || (str[i] == '-' && i > 0)) return
(false);
}
return (true);
}
// Returns 'true' if the string is a REAL NUMBER.
bool isRealNumber(char* str)
{
int i, len = strlen(str);
bool hasDecimal = false;
if (len == 0) return (false); for (i = 0; i <
len; i++) { if (str[i] != '0' && str[i] != '1' &&
str[i] != '2'
&& str[i] != '3' && str[i] != '4' && str[i] != '5'
&& str[i] != '6' && str[i] != '7' && str[i] != '8'
&& str[i] != '9' && str[i] != '.' ||
(str[i] == '-' && i > 0)) return
(false);
if (str[i] == '.')
hasDecimal = true;
}
return (hasDecimal);
}
// Extracts the SUBSTRING.
char* subString(char* str, int left, int right)
{ int
i;
char* subStr = (char*)malloc(
```

```
sizeof(char) * (right - left + 2)); for
(i = left; i <= right; i++) subStr[i -
left] = str[i]; subStr[right - left + 1]
= '\0'; return (subStr);
}
// Parsing the input STRING.
void parse(char* str)
{
int left = 0, right = 0; int len = strlen(str); while
(right <= len && left <= right) { if
(isDelimiter(str[right]) == false) right++; if
(isDelimiter(str[right]) == true && left == right) { if
(isOperator(str[right]) == true) printf("'%' IS AN
OPERATOR\n", str[right]); right++; left = right;
} else if (isDelimiter(str[right]) == true && left != right
|| (right == len && left != right)) { char*
subStr = subString(str, left, right - 1); if
(isKeyword(subStr) == true) printf("'%' IS
A KEYWORD\n", subStr); else if
(isInteger(subStr) == true) printf("'%' IS
AN INTEGER\n", subStr); else if
(isRealNumber(subStr) == true)
printf("'%' IS A REAL NUMBER\n",
subStr); else if (validIdentifier(subStr) ==
true && isDelimiter(str[right - 1]) == false)
printf("'%' IS A VALID IDENTIFIER\n",
subStr); else if (validIdentifier(subStr) ==
false && isDelimiter(str[right - 1]) == false)
```

```
printf("%s' IS NOT A VALID IDENTIFIER\n",
subStr);
left = right;
}
}
return;
}
// DRIVER FUNCTION
int main()
{
// maximum length of string is 100 here
char str[100] = "int a = b + 1c; ";
parse(str); // calling the parse function
return (0);
}
```

OUTPUT:

```
'int' IS A KEYWORD
'a' IS A VALID IDENTIFIER
'=' IS AN OPERATOR
'b' IS A VALID IDENTIFIER
'+' IS AN OPERATOR
'1c' IS NOT A VALID IDENTIFIER

...Program finished with exit code 0
Press ENTER to exit console.
```

Program – 3

Program for Lexical analysis using LEX tools

```
/* program name is lexp.l */
%{
/* program to recognize a c program */
int COMMENT=0;
int cnt=0; %}
identifier [a-zA-Z][a-zA-Z0-9]*
%%
#. * { printf("\n%s is a PREPROCESSOR DIRECTIVE",yytext);}
int | float | char | double | while | for | do |
if |
break |
continue |
void |
switch |
case | long
| struct |
const |
typedef |
return |
else |
goto {printf("\n\t%s is a KEYWORD",yytext);}
"/*" {COMMENT = 1;}
"*/" {COMMENT = 0; cnt++;}
{identifier}\( {if(!COMMENT)printf("\n\nFUNCTION\n\t%s",yytext);}
\{ {if(!COMMENT) printf("\n BLOCK BEGINS");}
\} {if(!COMMENT) printf("\n BLOCK ENDS");}
{identifier}\([0-9]*\)? {if(!COMMENT) printf("\n %s IDENTIFIER",yytext);}
\".*" {if(!COMMENT) printf("\n\t%s is a STRING",yytext);}
[0-9]+ {if(!COMMENT) printf("\n\t%s is a NUMBER",yytext);}
\\(\\;)? {if(!COMMENT) printf("\n\t");ECHO;printf("\n");}
\\( ECHO;
= {if(!COMMENT)printf("\n\t%s is an ASSIGNMENT OPERATOR",yytext);}
\\<= |
\\>= |
\\< |
== |
\\> {if(!COMMENT) printf("\n\t%s is a RELATIONAL OPERATOR",yytext);}
%%
```

```
int main(int argc,char **argv)
{
if (argc> 1)
{
FILE *file; file =
fopen(argv[1],"r");
if(!file)
{
printf("could not open %s \n",argv[1]);
exit(0);
} yyin =
file;
}
yylex();
printf("\n\n Total No.Of comments are %d",cnt); return
0;
}
int yywrap()
{
return 1;
}
```

INPUT:

```
//var.c
#include<stdio.h> #include<conio.h>
void main()
{
int a,b,c; a=1; b=2;
c=a+b;
printf("Sum:%d",c);
}
```

OUTPUT:

```
l2sys29@l2sys29-Veriton-M275:~/Desktop/syedvirus$ lex exp3_lex.l
l2sys29@l2sys29-Veriton-M275:~/Desktop/syedvirus$ cc lex.yy.c
l2sys29@l2sys29-Veriton-M275:~/Desktop/syedvirus$ ./a.out

#include<stdio.h> is a preprocessor directive
#include<conio.h> is a preprocessor directive
void is a keyword
FUNCTION
main(
)

BLOCK BEGINS
int is a keyword
a IDENTIFIER,
b IDENTIFIER,
c IDENTIFIER;

a IDENTIFIER
= is an ASSIGNMENT OPERATOR
1 is a NUMBER ;

b IDENTIFIER
= is an ASSIGNMENT OPERATOR
2 is a NUMBER ;

c IDENTIFIER
= is an ASSIGNMENT OPERATOR
a IDENTIFIER+
b IDENTIFIER;

FUNCTION
printf(
"Sum:%d" is a STRING,
c IDENTIFIER
)
;
BLOCK ENDS
```

Program – 4**Program to find FIRST in context free grammar**

```
// C program to calculate the First and
```

```
// Follow sets of a given grammar
```

```
#include<stdio.h>
```

```
#include<ctype.h>
```

```
#include<string.h>
```

```
// Functions to calculate Follow
```

```
void followfirst(char, int, int);
```

```
void follow(char c);
```

```
// Function to calculate First
```

```
void findfirst(char, int, int);
```

```
int count, n = 0;
```

```
// Stores the final result
```

```
// of the First Sets
```

```
char calc_first[10][100];
```

```
// Stores the final result
```

```
// of the Follow Sets
```

```
char calc_follow[10][100];
```

```
int m = 0;
```

```
// Stores the production rules
```

```
char production[10][10];
```

```
char f[10], first[10];
```

```
int k;
```

```
char ck;
```

```
int e;
```

```
int main(int argc, char **argv)
```



```
{
int jm = 0;
int km = 0;
int i, choice;
char c, ch;
count = 8;
// The Input grammar
strcpy(production[0], "E=TR");
strcpy(production[1], "R=+TR");
strcpy(production[2], "R=#");
strcpy(production[3], "T=FY");
strcpy(production[4], "Y=*FY");
strcpy(production[5], "Y=#");
strcpy(production[6], "F=(E)");
strcpy(production[7], "F=i");

int kay;
char done[count];

int ptr = -1;

// Initializing the calc_first array
for(k = 0; k < count; k++) {
for(kay = 0; kay < 100; kay++) {
calc_first[k][kay] = '!';
}
}

int point1 = 0, point2, xxx;
for(k = 0; k < count; k++)
{
c = production[k][0];
point2 = 0;
```

```
xxx = 0;
// Checking if First of c has
// already been calculated
for(kay = 0; kay <= ptr; kay++)
    if(c == done[kay])
        xxx = 1;
    if (xxx == 1)
        continue;
// Function call
findfirst(c, 0, 0);

ptr += 1;
// Adding c to the calculated list
done[ptr] = c;
printf("\n First(%c) = { ", c);
calc_first[point1][point2++] = c;

// Printing the First Sets of the grammar
for(i = 0 + jm; i < n; i++) {
    int lark = 0, chk = 0;
    for(lark = 0; lark < point2; lark++) {
        if (first[i] == calc_first[point1][lark])
        {
            chk = 1;
            break;
        }
    }
    if(chk == 0)
    {
        printf("%c, ", first[i]);
```

```
calc_first[point1][point2++] = first[i];
}
}
printf("\n");
jm = n;
point1++;
}
printf("\n");
printf(".....\n\n");
char donee[count];

ptr = -1;
// Initializing the calc_follow array
for(k = 0; k < count; k++) {
    for(kay = 0; kay < 100; kay++) {
        calc_follow[k][kay] = '!';
    }
}
point1 = 0;
int land = 0;

for(e = 0; e < count; e++)
{
    ck = production[e][0];
    point2 = 0;
    xxx = 0;
    // Checking if Follow of ck
    // has already been calculated
    for(kay = 0; kay <= ptr; kay++)
        if(ck == donee[kay])
            xxx = 1;
```

```
if (xxx == 1)
continue;

land += 1;

// Function call
follow(ck);

ptr += 1;

// Adding ck to the calculated list
donee[ptr] = ck;

printf(" Follow(%c) = { ", ck);

calc_follow[point1][point2++] = ck;

// Printing the Follow Sets of the grammar
for(i = 0 + km; i < m; i++) {

int lark = 0, chk = 0;

for(lark = 0; lark < point2; lark++)

{

if (f[i] == calc_follow[point1][lark])

{

chk = 1;

break;

}

}

if(chk == 0)

{

printf("%c, ", f[i]);

calc_follow[point1][point2++] = f[i];

}

}

printf(" }\n\n");

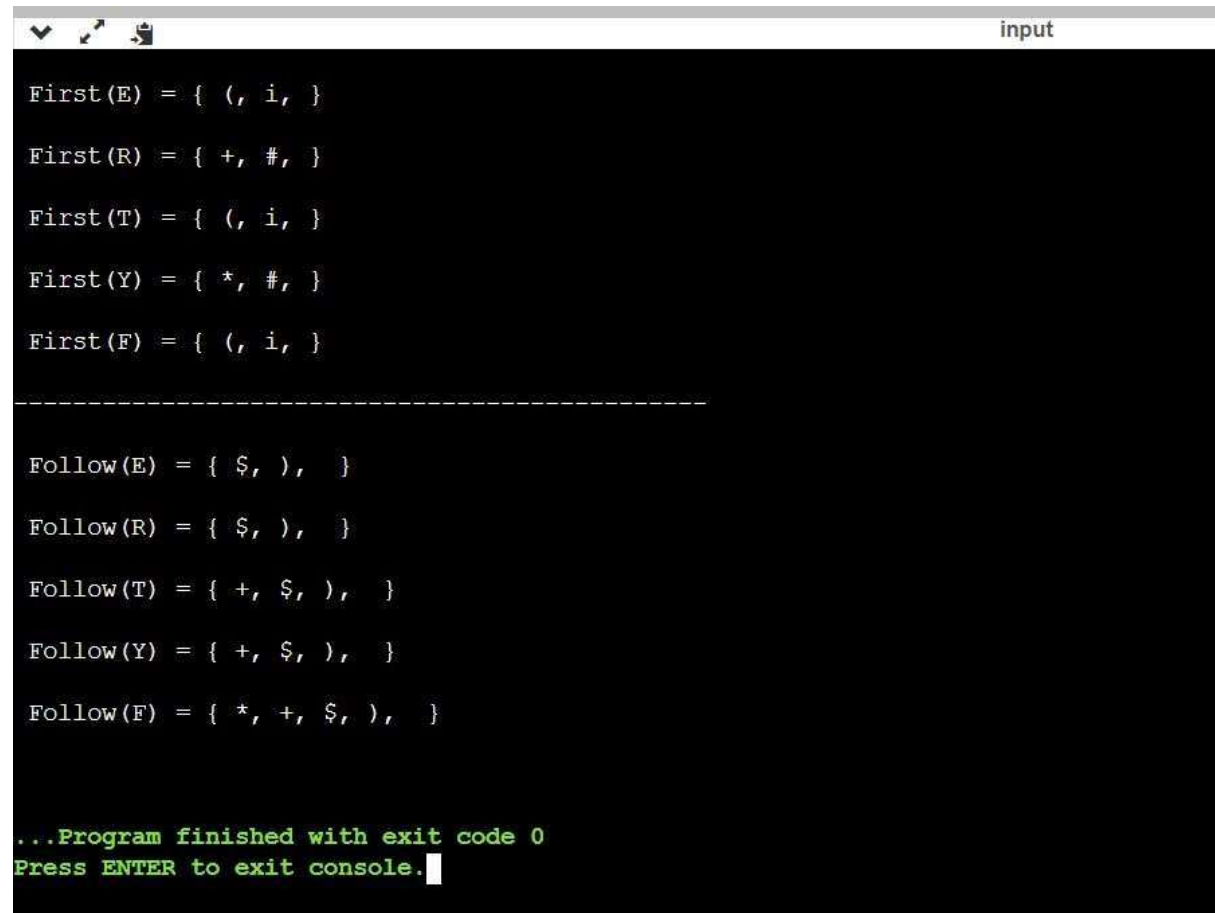
km = m; point1++;
```

```
}  
}  
void follow(char c)  
{ int i, j;  
  // Adding "$" to the follow  
  // set of the start symbol  
  if(production[0][0] == c) {  
    f[m++] = '$';  
  }  
  for(i = 0; i < 10; i++)  
  {  
    for(j = 2; j < 10; j++)  
    {  
      if(production[i][j] == c)  
      {  
        if(production[i][j+1] != '\0')  
        {  
          // Calculate the first of the next  
          // Non-Terminal in the production  
          followfirst(production[i][j+1], i, (j+2));  
        }  
        if(production[i][j+1] == '\0' && c != production[i][0])  
        {  
          // Calculate the follow of the Non-Terminal  
          // in the L.H.S. of the production  
          follow(production[i][0]);  
        }  
      }  
    }  
  }  
}
```

```
}  
}  
void findfirst(char c, int q1, int q2)  
{ int j;  
  // The case where we  
  // encounter a Terminal  
  if(!isupper(c)) {  
    first[n++] = c;  
  }  
  for(j = 0; j < count; j++)  
  {  
    if(production[j][0] == c)  
    {  
      if(production[j][2] == '#')  
      {  
        if(production[q1][q2] == '\0')  
        first[n++] = '#';  
        else if(production[q1][q2] != '\0'  
        && (q1 != 0 || q2 != 0))  
        {  
          // Recursion to calculate First of New  
          // Non-Terminal we encounter after epsilon  
          findfirst(production[q1][q2], q1, (q2+1));  
        }  
        else  
        first[n++] = '#';  
        else if(!isupper(production[j][2]))  
        {  
          first[n++] = production[j][2];  
        }  
      }  
    }  
  }  
}
```

```
}  
else  
{  
    // Recursion to calculate First of  
    // New Non-Terminal we encounter  
    // at the beginning  
    findfirst(production[j][2], j, 3);  
}  
}  
}  
}  
void followfirst(char c, int c1, int c2)  
{  
    int k;  
    // The case where we encounter  
    // a Terminal  
    if(!(isupper(c)))  
        f[m++] = c;  
    else  
    {  
        int i = 0, j = 1;  
        for(i = 0; i < count; i++)  
        {  
            if(calc_first[i][0] == c)  
                break;  
  
            //Including the First set of the  
            // Non-Terminal in the Follow of  
            // the original query
```

```
while(calc_first[i][j] != '!')
{
    if(calc_first[i][j] != '#')
    {
        f[m++] = calc_first[i][j];
    }
    else
    {
        if(production[c1][c2] == '\0')
        {
            // Case where we reach the
            // end of a production
            follow(production[c1][0]);
        }
        else
        {
            // Recursion to the next symbol
            // in case we encounter a "#"
            followfirst(production[c1][c2], c1, c2+1);
        }
    }
    j++;
}
```


OUTPUT:

```
input

First(E) = { (, i, }
First(R) = { +, #, }
First(T) = { (, i, }
First(Y) = { *, #, }
First(F) = { (, i, }

-----

Follow(E) = { $, ), }
Follow(R) = { $, ), }
Follow(T) = { +, $, ), }
Follow(Y) = { +, $, ), }
Follow(F) = { *, +, $, ), }

...Program finished with exit code 0
Press ENTER to exit console.
```

Program-5

Program to find whether a given string is keyword or not

```
#include <stdio.h>
#include <string.h>
int
main() { char
keyword[32][10]={
    "auto","double","int","struct","break","else","long",
    "switch","case","enum","register","typedef","char",
    "extern","return","union","const","float","short",
    "unsigned","continue","for","signed","void","default",
    "goto","sizeof","volatile","do","if","static","while"
}
;
char str[]="which";
int flag=0,i; for(i = 0; i < 32;
i++) {
if(strcmp(str,keyword[i])==0) {
    flag=1;
}
}
if(flag==1) printf("%s is a
keyword",str);
else
    printf("%s is not a keyword",str);
}
```

OUTPUT:A screenshot of a compiler's output window. The window has a title bar with the word "input" on the right. The background is black with white and green text. The text displayed is: "which is not a keyword" on the first line, "...Program finished with exit code 0" on the second line, and "Press ENTER to exit console." on the third line. There are some small icons in the top left corner of the window.

```
which is not a keyword
...Program finished with exit code 0
Press ENTER to exit console.
```

Program – 6

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
#include<ctype.h>

void main()
{
    int i,flag;
    char a[5];

    clrscr();

    puts("Enter the value :: ");
    gets(a);

    for(i=0;i<strlen(a);i++)
    {
        if(isdigit(a[i]))
            flag=1;

        else
        {
            flag=0;
            break;
        }
    }

    if(flag==1)
        puts("Value is constant");
    else
        puts("Value is a variable");

    getch();
}
```

Output

Enter the value ::

123

Value is constant

Enter the value ::

vikas

Value is a variable

Program – 7**Program to identify whether a given line is a comment or not**

```
#include<stdio.h>

#include<conio.h> void
main()
{
    char text[30];
    int i = 2, a = 0;

    clrscr();
    printf("\nWrite a C Program to Identify Whether a Given Line is a Commentor Not.");
    printf("\n\nEnter Text : ");
    gets(text);    if(text[0] == '/')
    {
        if(text[1] == '/') {
            printf("\nIt is a Comment.");
        }
        else if(text[1] == '*')
        {
            for(i = 2; i <= 30; i++)
            {
                if(text[i] == '*' && text[i + 1] == '/')
                {
                    printf("\nIt is a Comment.");
                    a = 1;
                }
            }
        }
        break;
    }
}
```

```
        }
    else {
        continue;
    }
}

    if(a == 0)
    {
        printf("\nIt is Not a Comment.");
    }
    else
    {
        printf("\nIt is Not a Comment.");
    }
}

else
{
    printf("\nIt is Not a Comment.");
}

getch();
}
```

OUTPUT:

```
Write a C Program to Identify Whether a Given Line is a Comment or Not.
```

```
Enter Text : codejugaad
```

```
It is Not a Comment._
```

```
Write a C Program to Identify Whether a Given Line is a Comment or Not.
```

```
Enter Text : // codejugaad
```

```
It is a Comment._
```


Program – 8**Program to check whether a given identifier is valid or not**

```
#include <stdio.h>
#include <conio.h> void
main()
{
int i=0,flag=0;
char
keyw[10][10]={"int","float","break","long","char","for","if","switch","else","while"},a[10];
clrscr();
printf("Enter Identifier : ");
gets(a);   for(i=0;i<10;i++)
{
    if((strcmp(keyw[i],a)==0))
    {
        flag=1;
    }
}
if(flag==1)
{
    printf("\n%s is Keyword.",a);
}
else
{
    flag=0;
```

```
        if((a[0]=='_') || (isalpha(a[0])!=0))
        {
            for(i=1;a[i]!='\0';i++)
            {
                if((isalnum(a[i])==0)&&(a[i]!='_'))
                {
flag=1;
                }
            }
        }
        else
        {
            flag=1;
        }
    }
    if(flag==0)
    {
        printf("\n%s is an Identifier.",a);
    }
    else
    {
        printf("\n%s is Not an Identifier.",a);
    }
    getch();
}
```

OUTPUT:

```
Enter Identifier : _id
```

```
_id is an Identifier._
```

```
Enter Identifier : identi
```

```
identi is an Identifier._
```

```
Enter Identifier : int
```

```
int is Keyword.
```

```
Enter Identifier : 1_id
```

```
1_id is Not an Identifier.
```

Program – 9

Program to recognize strings under 'a', 'a*b+', 'abb'

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
#include<stdlib.h> void
main()
{
char s[20],c; int state=0,i=0;
clrscr(); printf("\n Enter a
string:"); gets(s); while(s[i]!='\0')
{
switch(state)
{
case 0: c=s[i++]; if(c=='a')
state=1;
else if(c=='b') state=2;
else state=6; break;
case 1: c=s[i++];
if(c=='a')
state=3;
else if(c=='b')
state=4; else
break; case 2: c=s[i++]; state=6;
```

```
if(c=='a') state=6;
else if(c=='b') state=2;
else
break; state=6; case
3: c=s[i++]; if(c=='a')
state=3; else if(c=='b')
state=2;

else break; case 4:
c=s[i++]; state=6;
if(c=='a')
state=6;

else if(c=='b') state=5;
else break; case 5:
c=s[i++];
if(c=='a')
state=6; state=6;

else if(c=='b')
state=2; else state=6; break; case 6: printf("\n %s
is not recognised.",s); exit(0); }

}

if(state==1)
printf("\n %s is accepted under rule 'a'",s); else if((state==2) || (state==4))
printf("\n %s is accepted under rule 'a*b+'",s); else if(state==5) printf("\n
%s is accepted under rule 'abb'",s); getch();
}
```

OUTPUT:

Enter a String: aaaabbbbb

aaaabbbbb is accepted under rule 'a*b+'

Enter a string: cdgs

cdgs is not recognized

Program – 10

Program to simulate lexical analyser for validating operations

```
#include<stdio.h>

#include<conio.h> void

main()

{

char s[5]; clrscr();

printf("\n Enter any operator:");

gets(s); switch(s[0])

{

case '>':

if(s[1]=='=')

printf("\n Greater than or equal");

else printf("\n Greater than");

break; case '<':

if(s[1]=='=') printf("\n Less

than or equal"); else

printf("\nLess than"); break;

case '=':

if(s[1]=='=')

printf("\nEqual to"); else

printf("\nAssignment");

break;

case '!': if(s[1]=='=')
```

```
printf("\nNot Equal"); else
printf("\n Bit Not"); break;
case '&': if(s[1]=='&')
printf("\nLogical AND"); else
printf("\n Bitwise AND");
break;
case '|': if(s[1]=='|')
printf("\nLogical OR");
else printf("\nBitwise
OR"); break; case '+':
printf("\n Addition");
break;
case '-':
printf("\nSubstraction");
break; case '*':
printf("\nMultiplication");
break;
case '/':
printf("\nDivision"); break;
case '%':
printf("Modulus");
break; default:
printf("\n Not a operator");
}
getch();
}
```


OUTPUT:

```
Enter any operator:7
```

```
Not a operator_
```

```
Enter any operator:!!
```

```
Logical OR
```

Program – 11

Program for implementation of Operator Precedence Parser

```
#include<stdio.h>

#include<string.h> char

*input;

int i=0;

char lasthandle[6],stack[50],handles[][5]={"}E(", "E*E", "E+E", "i", "E^E"};
//(E) becomes )E( when pushed to stack

int top=0,l; char

prec[9][9]={

/*input*/

/*stack  +  -  *  /  ^  i  (  )  $  */

/* + */ '>','>','<','<','<','<','<','<','>','>',

/* - */ '>','>','<','<','<','<','<','<','>','>',

/* * */ '>','>','>','>','<','<','<','<','>','>',

/* / */ '>','>','>','>','<','<','<','<','>','>',

/* ^ */ '>','>','>','>','<','<','<','<','>','>',
```

```
/* i */ '>', '>', '>', '>', '>', 'e', 'e', '>', '>',
```

```
/* ( */ '<', '<', '<', '<', '<', '<', '<', '>', 'e',
```

```
/* ) */ '>', '>', '>', '>', '>', 'e', 'e', '>', '>',
```

```
/* $ */ '<', '<', '<', '<', '<', '<', '<', '<', '>',
```

```
};
```

```
int getIndex(char c)
```

```
{
```

```
switch(c)
```

```
{
```

```
case '+':return 0;
```

```
case '-':return 1;
```

```
case '*':return 2;
```

```
case '/':return 3;
```

```
case '^':return 4;
```

```
case 'i':return 5; case
```

```
('':return 6; case
```

```
')':return 7; case
```

```
'$':return 8;
```

```
}
```

```
}
```

```
int shift()
```

```
{
```

```
stack[++top]=*(input+i++); stack[top+1]='\0';
```

```
}
```

```
int reduce()
```

```
{
```

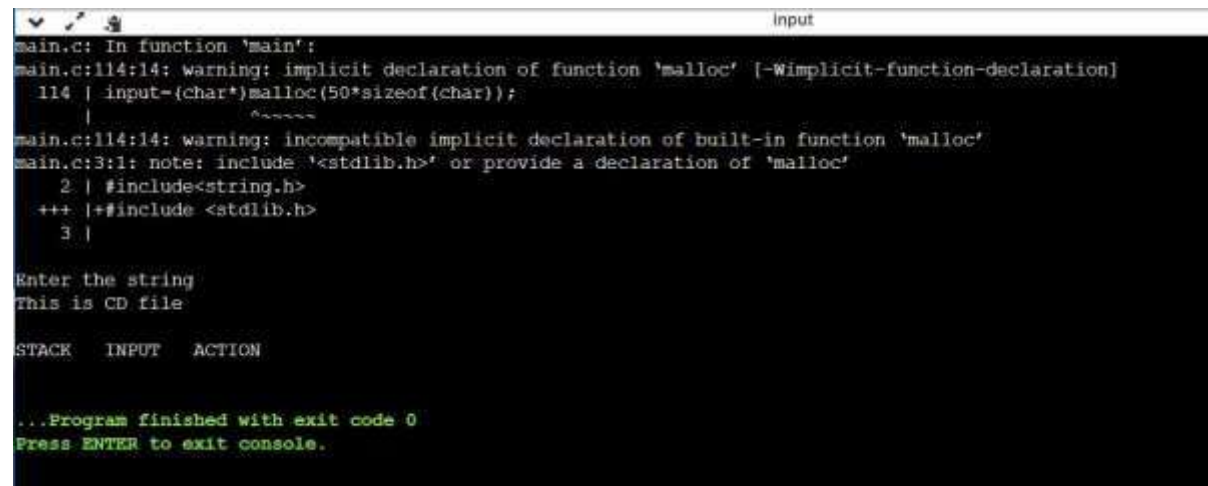
```
int i,len,found,t; for(i=0;i<5;i++)//selecting
handles {

len=strlen(handles[i]); if(stack[top]==handles[i][0]&&top+1>=len)
{
found=1;
for(t=0;t<len;t++)
{
if(stack[top-t]!=handles[i][t])
{
found=0; break;
}
}
if(found==1)
{
stack[top-t+1]='E'; top=top-
t+1;
strcpy(lasthandle,handles[i]);
stack[top+1]='\0'; return
1;//successful reduction
}
}
}
return 0;
}

void dispstack()
{ int
j;
for(j=0;j<=top;j++)
printf("%c",stack[j]);
```

```
}  
void dispinput()  
{ int  
j;  
for(j=i;j<l;j++) printf("%c",*(input+j));  
}  
void main()  
{ int  
j;  
input=(char*)malloc(50*sizeof(char));  
printf("\nEnter the string\n");  
scanf("%s",input);  
input=strcat(input,"$");  
l=strlen(input); strcpy(stack,"$");  
printf("\nSTACK\tINPUT\tACTION");  
while(i<=l)  
{  
shift(); printf("\n"); dispstack(); printf("\t");  
dispinput(); printf("\tShift");  
if(prec[getindex(stack[top])][getindex(input[i])]=='>')  
{  
while(reduce())  
{  
printf("\n"); dispstack(); printf("\t");  
dispinput(); printf("\tReduced: E-  
>%s",lasthandle);  
}  
}  
}
```

```
if(strcmp(stack,"$E$")==0)
printf("\nAccepted;");
else printf("\nNot
Accepted;");
}
```

OUTPUT:

```
main.c: In function 'main':
main.c:114:14: warning: implicit declaration of function 'malloc' [-Wimplicit-function-declaration]
  114 | input=(char*)malloc(50*sizeof(char));
      |             ^~~~~~
main.c:114:14: warning: incompatible implicit declaration of built-in function 'malloc'
main.c:3:1: note: include '<stdlib.h>' or provide a declaration of 'malloc'
   2 | #include<string.h>
   +-+ |+#include <stdlib.h>
   3 |

Enter the string
This is CD file

STACK  INPUT  ACTION

...Program finished with exit code 0
Press ENTER to exit console.
```

Program – 12A

Program for implementation of calculator using YACC tool

```
%{  
/* Definition section */  
  
#include<stdio.h>  
  
#include "y.tab.h" extern  
  
int yylval;  
  
%}  
  
/* Rule Section */  
  
%%  
  
[0-9]+ { yylval=atoi(yytext);  
return NUMBER;  
}  
  
[\t];  
  
[\n] return 0;  
  
. return yytext[0];  
  
%%  
  
int yywrap()  
{  
return 1;  
}
```

Parser code:

```
%{
```



```
/* Definition section */
#include<stdio.h>

int flag=0;

%}

%token NUMBER

%left '+' '-'

%left '*' '/' '%'

%left '(' ')'

/* Rule Section */

%%

ArithmeticExpression:    E{
printf("\nResult=%d\n", $$);
return 0;
};

E:E+'E' {$$=$1+$3;}
|E-'E' {$$=$1-$3;}
|E'*E' {$$=$1*$3;}
|E'/E' {$$=$1/$3;}
|E'%E' {$$=$1%$3;}
| '('E' {$$=$2;}
| NUMBER {$$=$1;} ;

%%

//driver code void
main()
{
printf("\nEnter Any Arithmetic Expression which can have operations Addition,
Subtraction, Multiplication, Division, Modulus and
Round brackets:\n"); yyparse(); if(flag==0)
```

```
printf("\nEntered arithmetic expression is Valid\n\n");  
}
```

```
void yyerror()
```

```
{  
printf("\nEntered arithmetic expression is Invalid\n\n"); flag=1;  
}
```

OUTPUT:

```
thakur@thakur-VirtualBox:~/Documents/new$ lex calc.l
thakur@thakur-VirtualBox:~/Documents/new$ yacc calc.y
calc.y:39 parser name defined to default : "parse"
thakur@thakur-VirtualBox:~/Documents/new$ gcc lex.yy.c y.tab.c -w
thakur@thakur-VirtualBox:~/Documents/new$ ./a.out

Enter Any Arithmetic Expression which can have operations Addition, Subtraction, Multiplication, Division, Modulus and Round brackets:
4+5

Result=9

Entered arithmetic expression is Valid

thakur@thakur-VirtualBox:~/Documents/new$ ./a.out

Enter Any Arithmetic Expression which can have operations Addition, Subtraction, Multiplication, Division, Modulus and Round brackets:
10-5

Result=5

Entered arithmetic expression is Valid

thakur@thakur-VirtualBox:~/Documents/new$ ./a.out

Enter Any Arithmetic Expression which can have operations Addition, Subtraction, Multiplication, Division, Modulus and Round brackets:
10+5

Entered arithmetic expression is Invalid

thakur@thakur-VirtualBox:~/Documents/new$ ./a.out

Enter Any Arithmetic Expression which can have operations Addition, Subtraction, Multiplication, Division, Modulus and Round brackets:
10/5

Result=2

Entered arithmetic expression is Valid

thakur@thakur-VirtualBox:~/Documents/new$ ./a.out

Enter Any Arithmetic Expression which can have operations Addition, Subtraction, Multiplication, Division, Modulus and Round brackets:
(2+5)*3

Result=21

Entered arithmetic expression is Valid

thakur@thakur-VirtualBox:~/Documents/new$ ./a.out

Enter Any Arithmetic Expression which can have operations Addition, Subtraction, Multiplication, Division, Modulus and Round brackets:
(2*4)+

Entered arithmetic expression is Invalid

thakur@thakur-VirtualBox:~/Documents/new$ ./a.out

Enter Any Arithmetic Expression which can have operations Addition, Subtraction, Multiplication, Division, Modulus and Round brackets:
2%5

Result=2

Entered arithmetic expression is Valid
```

Program – 12B

Program for implementation of Recursive Descent Parser using LEX tool

```
#include<stdio.h>
#include<ctype.h>
#include<string.h>
void Tprime();
void Eprime();
void E(); void
check(); void T();
char expression[10];
int count, flag; int
main()
{
count = 0;
flag = 0;
printf("\nEnter an Algebraic Expression:\t"); scanf("%s",
expression);
E();
if((strlen(expression) == count) && (flag == 0))
{
printf("\nThe Expression %s is Valid\n", expression);
}
else
{
printf("\nThe Expression %s is Invalid\n", expression);
}
}
```

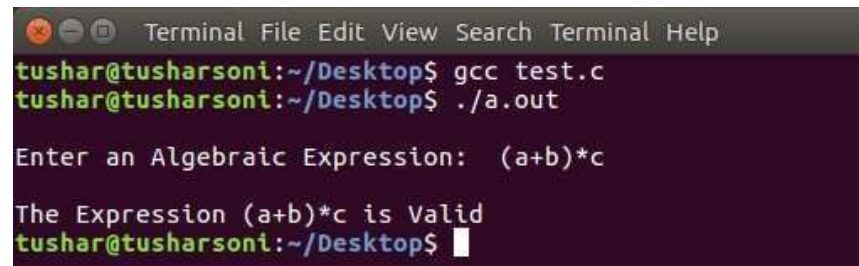
```
void E()
{
    T();
    Eprime();
}

void T()
{
    check();
    Tprime();
}

void Tprime()
{
    if(expression[count] == '*')
    {
        count++; check();
        Tprime();
    }
}

void check()
{
    if(isalnum(expression[count]))
    {
        count++;
    }
    else if(expression[count] == '(')
    {
        count++;
        E();
    }
    if(expression[count] == ')')
```

```
{
count++;
}
else
{
flag = 1;
}
}
else
{
flag = 1;
}
}
void Eprime()
{
if(expression[count] == '+')
{
count++;
T();
Eprime();
}
}
```

OUTPUT:

```
Terminal File Edit View Search Terminal Help
tushar@tusharsoni:~/Desktop$ gcc test.c
tushar@tusharsoni:~/Desktop$ ./a.out

Enter an Algebraic Expression: (a+b)*c

The Expression (a+b)*c is Valid
tushar@tusharsoni:~/Desktop$
```

Program – 13

Program for implementation of LALR Parser

```
else
{
if(temp[0]=='r')
{
j=is#include<stdio.h>
#include<ctype.h>
#include<string.h> void
push(char *,int *,char);
char stacktop(char *); void
isproduct(char,char);
int ister(char); int isinter(char); int
isstate(char); void error(); void
isreduce(char,char); char
pop(char *,int *); void printt(char
*,int *,char [],int); void rep(char
[],int); struct action
{
char row[6][5];
}
;
const struct action A[12]={
{"sf","emp","emp","se","emp","emp"},
{"emp","sg","emp","emp","emp","acc"}, {"emp","rc","sh","emp","rc","rc"},
{"emp","re","re","emp","re","re"},
{"sf","emp","emp","se","emp","emp"},
```



```
{"emp","rg","rg","emp","rg","rg"},
{"sf","emp","emp","se","emp","emp"},
{"sf","emp","emp","se","emp","emp"},
{"emp","sg","emp","emp","sl","emp"},
{"emp","rb","sh","emp","rb","rb"}, {"emp","rb","rd","emp","rd","rd"},
{"emp","rf","rf","emp","rf","rf"}
};

struct gotol
{
char r[3][4];
};

const struct gotol G[12]={ {"b","c","d"},
{"emp","emp","emp"},
{"emp","emp","emp"},
{"emp","emp","emp"},
{"i","c","d"},
{"emp","emp","emp"},
{"emp","j","d"},
{"emp","emp","k"},
{"emp","emp","emp"},
{"emp","emp","emp"},
}
;

char ter[6]={ 'i','+', '*', ')', '(', '$' };
char nter[3]={ 'E', 'T', 'F' } char
states[12]={ 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'm', 'j', 'k', 'l' };
char stack[100];
int top=-1; char
```

```
temp[10];
struct grammar
{
char left; char
right[5];
}
;
const struct grammar rl[6]={
{'E',"e+T"}, {'E',"T"},
{'T',"T*F"},
{'T',"F"},
{'F'," (E)"},
{'F'," i"},
}
;
void main()
{
char inp[80],x,p,dl[80],y,bl='a';
int i=0,j,k,l,n,m,c,len; clrscr();
printf(" Enter the input :");
scanf("%s",inp);
len=strlen(inp); inp[len]='$';

inp[len+1]='\0';
push(stack,&top,bl);
printf("\n stack \t\t\t input");
printt(stack,&top,inp,i); do
{
x=inp[i];
```

```
p=stacktop(stack); isproduct(x,p);
if(strcmp(temp,"emp")==0)
error();
if(strcmp(temp,"acc")==0)
break; else
{
if(temp[0]=='s')
{
push(stack,&top,inp[i]); push(stack,&top,temp[1]);
i++;
}
state(temp[1]);
strcpy(temp,rl[j-2].right);
dl[0]=rl[j-2].left; dl[1]='\0';
n=strlen(temp); for(k=0;k<2*n;k++)
pop(stack,&top);
for(m=0;dl[m]!='\0';m++)
push(stack,&top,dl[m]); l=top;
y=stack[l-1]; isreduce(y,dl[0]);
for(m=0;temp[m]!='\0';m++) push(stack,&top,temp[m]);
}
}
}
printt(stack,&top,inp,i); }while(inp[i]!='\0');
if(strcmp(temp,"acc")==0) printf(" \n
accept the input "); else printf(" \n
do not accept the input "); getch();
}
void push(char *s,int *sp,char item)
```

```
{
if(*sp==100)
printf(" stack is full "); else
{
*sp=*sp+1; s[*sp]=item;
}
}

char stacktop(char *s)
{
char i; i=s[top];
return i;
}

void isproduct(char x,char p)
{
int k,l;
k=ister(x); l=isstate(p);
strcpy(temp,A[l-1].row[k-1]);
}

int ister(char x)
{ int
i;
for(i=0;i<6;i++)
if(x==ter[i])
return i+1; return
0;
}

int isnter(char x)
{ int
i;
```

```
for(i=0;i<3;i++)
if(x==nter[i])
return i+1; return
0;
}
int isstate(char p)
{
int i;
for(i=0;i<12;i++)
if(p==states[i])
return i+1; return
0;
}
void error()
{
printf(" error in the input "); exit(0);
}
void isreduce(char x,char p)
{
int k,l;
k=isstate(x); l=isnter(p);
strcpy(temp,G[k-1].r[l-1]);
}
char pop(char *s,int *sp)
{
char item; if(*sp== -1)
printf(" stack is empty ");
else
{
```

```
item=s[*sp]; *sp=*sp-1;
}
return item;
}

void printt(char *t,int *p,char inp[],int i)
{
int r;
printf("\n"); for(r=0;r<=*p;r++)
rep(t,r); printf("\t\t\t");
for(r=i;inp[r]!='\0';r++)
printf("%c",inp[r]);
}

void rep(char t[],int r)
{
char c; c=t[r];
switch(c)
{
case 'a': printf("0"); break;
case 'b': printf("1"); break;
case 'c': printf("2");
break;
case 'd': printf("3"); break;
case 'e': printf("4"); break;
case 'f': printf("5");
break;
case 'g': printf("6"); break;
case 'h': printf("7");
break; case 'm':
printf("8"); break;
```

```
case 'j': printf("9");  
break; case 'k':  
printf("10"); break;  
case 'l': printf("11");  
break; default  
:printf("%c",t[r]); break;  
}  
}
```

[illegible]